

Bridging Misconception Detection and Dialogue-Based Tutoring for Enhanced Learning in Programming Education

Anonymous ACL submission

Abstract

Students learning programming often develop misconceptions that make learning difficult. While recent work shows that large language models (LLMs) can reduce instructional workload (Tian et al., 2025), it remains unclear whether they effectively support student learning. Current AI tutoring systems do not fully address this issue: some provide step-by-step guidance, while others detect mistakes in code, but they do not combine both in an effective way. In this work, we combine two existing approaches: misconception detection (Al-Hossami and Bunescu, 2026) and dialogue-based tutoring (Wang et al., 2025). Our system first identifies potential misconceptions and then guides students through interactive dialogue to help them understand and correct their mistakes. Our work highlights the potential of integrating misconception detection with dialogue-based tutoring to better support conceptual understanding in programming education. By focusing on helping students recognize and correct their own misunderstandings, our approach aims to provide a more effective alternative to answer-driven tutoring systems.¹

1 Introduction

The role of LLMs in education has shifted from something that needs to be restricted to something that should be integrated. Recent work shows that LLMs can meaningfully reduce teacher workload (Tian et al., 2025), but whether students actually learn from conversations led by an LLM tutor remains an open question (Francis et al., 2025). This question is especially pressing in introductory programming courses, where students frequently develop persistent misconceptions—false beliefs about language semantics that cause bugs, hinder progress on related concepts, and accumulate throughout a course (Al-Hossami and Bunescu,

2026). Educators do not want students to bypass their misunderstandings by receiving direct answers from generative AI, as this prevents them from building a solid foundation.

Existing LLM-based tutoring systems for coding largely sidestep this problem. Dialogue-based tutors such as TRAVER (Wang et al., 2025) model a student’s knowledge state as a set of missing knowledge components and guide them toward filling those gaps. However, a missing concept and an actively incorrect belief are different failure modes: a student who believes $range(n)$ produces values from 1 to n will not be helped by being told the correct steps. Instead, they need the misconception itself surfaced and corrected. Conversely, McMiner (Al-Hossami and Bunescu, 2026) can mine misconceptions from student code, but it offers no pedagogical path afterwards once one misconception is found.

Therefore, we present our combination of the misconception detection tool (Al-Hossami and Bunescu, 2026) with the dialogue-based tutor system (Wang et al., 2025) specifically for students who are learning how to code. Our workflow aims to guide students to identify their own misconceptions, leading them to recognize the false belief before progressing toward task completion.

2 Related Work

In this section, we briefly discuss the existing work and benchmarks.

2.1 Misconception Detection in Programming

Identifying where and why students go wrong in code has a long history in CS education research. Early systems such as PROUST (Johnson and Soloway, 1984) and later rule-based tools relied on hand-crafted patterns targeting a small set of known misconceptions. A common limitation is that these approaches either operate over a fixed

¹Github: [Link](#), Hugging Face: [Link](#).

taxonomy (Brown and Altadmri, 2017), or they simply flag the bug without articulating the underlying false belief.

McMining addresses this gap by framing misconception discovery as a generative task: given a bag of problem-code pairs from a student, an LLM-based miner produces a natural-language description of the misconception the code exhibits, including novel misconceptions outside any predefined bank. We build on McMining as the diagnostic component of our pipeline, using its output as a grounding signal for downstream remediation.

2.2 Dialogue-Based Task Tutoring with LLMs

LLM-powered tutoring agents have been explored across many educational domains, including language learning (Kwon et al., 2024), math (Wang et al., 2024), and science (Zhang et al., 2024). However, most of this work targets knowledge delivery that helps students understand a specific concept, and evaluates success through closed-form assessments such as question answering or multiple-choice tests.

Coding tutoring is a harder regime: tasks are open-ended, require understanding cross-file dependencies, and admit many valid solutions, thus making it harder to evaluate (Wang et al., 2025). Early LLM-based code tutors such as CHAT-TUTOR (Chen et al., 2024) primarily plays a reactive role, answering student questions or clarifying doubts on demand. TreeInstruct (Zhao et al., 2024) moves toward proactive guidance using Socratic tree-based questioning for code debugging, but does not explicitly model task completion as the tutoring objective.

TRAVER (Wang et al., 2025) addresses this by combining knowledge tracing with a turn-by-turn verifier that scores candidate tutor utterances against the likelihood of eventual task completion.

3 Dataset

Our primary dataset is based on the TRAVER framework introduced by Wang et al. (2025), which is evaluated on the EvoCodeBench benchmark (Li, 2024). In this setting, coding tutoring is modeled as an interactive dialogue between a tutor and a student, centered around a target coding task, task-specific knowledge (e.g., dependencies and solution steps), and multi-turn interactions. The dataset includes repository-level coding tasks, associated knowledge components, and simulated

tutor-student dialogues.

We use this dataset in two ways. First, it serves as our **baseline setting**, where the tutor operates under the original TRAVER framework without explicit access to misconception information. In this baseline, the tutor relies only on knowledge tracing and dialogue context to guide the student (Wang et al., 2025).

To improve upon this baseline, we augment the dataset using McMiner (Al-Hossami and Bunesco, 2026). McMiner is trained and evaluated on the **McMining benchmark dataset**, which contains misconception-labeled problem-code pairs constructed via misconception injection. We use McMiner’s outputs to enrich the original TRAVER data with explicit misconception annotations.

Specifically, for each tutoring instance, we take the student-generated code from EvoCodeBench tasks and pass it through McMiner to extract a potential misconception. This information is then added as an additional field in the dataset. As a result, each data instance is represented as:

(problem description, student code, detected misconception, tutor response)

By incorporating misconception information, the tutor is no longer limited to identifying missing knowledge components, as in the baseline TRAVER framework. Instead, the tutor is explicitly informed about the student’s incorrect beliefs and can guide the student toward recognizing and correcting them. This enables more targeted and concept-focused feedback rather than purely procedural guidance.

We also include negative examples where McMiner outputs NONE, indicating that no clear misconception is detected. These cases are important because not all incorrect or suboptimal code reflects a misconception. In such scenarios, the tutor follows the baseline TRAVER behavior and focuses on guiding the student without assuming a specific conceptual error.

Overall, our dataset combines EvoCodeBench-based tutoring tasks with McMining-style misconception annotations, enabling a direct comparison between the baseline TRAVER tutor and our misconception-aware tutor.

4 Methodology

We follow the original authors’ approach to run the baseline. However, we made several modifications due to resource constraints.

4.1 Model and Data constraints

TRAYER used Mixtral-8x7B, which is a Sparse Mixture-of-Experts model, for the student agent. However, it is no longer available via hosted inference APIs, and it requires $\sim 90\text{GB}$ of VRAM which exceeds our available storage budget. To avoid potential data contamination with the coding benchmark, the original authors require the student agent’s model to have a release date prior to October 2023. Hence, we substitute Mistral-7B for the student model, as it satisfies this constraint.

Beyond the model choice, we also adjusted the benchmark scope. The benchmark EvoCodeBench has 25 repositories, and each with its own environment setup and dependencies (Li, 2024). Thus, we select a subset of 4 repositories for our experiments, as each virtual environment requires approximately 5GB of storage. Table 1 summarizes the differences between our choice and the paper’s choice.

4.2 Implementation Details

Our pipeline combines TRAYER’s tutor–student dialogue framework with McMining’s misconception extraction, with the goal of improving tutoring quality by surfacing student misconceptions and conditioning the tutor on them. The pipeline proceeds in three stages: baseline dialogue generation, misconception mining, and misconception-conditioned re-tutoring.

Baseline dialogue generation. Following TRAYER, we instantiate a tutor agent (Llama 3.1 70B) and a student agent (Mistral 7B) and run multi-turn tutoring dialogues on each selected EvoCodeBench task. Each dialogue is capped at 8 turns, after which the student agent produces k candidate solutions. We evaluate these against the task’s test suite using $\text{pass}@k$ and $\text{recall}@k$, following the original TRAYER evaluation protocol. This constitutes our baseline.

Misconception mining. We apply McMiner (Al-Hossami and Bunescu, 2026) to the outputs of TRAYER, using the generated student code and accompanying round descriptions from each dialogue round as input. McMiner produces natural-language candidate misconception statements characterizing possible reasoning errors reflected in the student code and its associated context. In our experiments, we use the local WAVE deployment of Mistral-7B-Instruct-v0.2 as the mining model, rather than Gemini. The resulting mis-

Setting	TRAYER	Ours
Student (dialogue)	Mixtral-8x7B	Mistral-7B
Student (codegen)	Llama-3.1-8B	Llama-3.1-8B
Tutor	Llama-3.1-70B	Llama-3.1-70B
Verifier	Mistral-7B+LoRA	Mistral-7B+LoRA
Repositories	25	4
Tasks	250	22
Quantization	FP16	4-bit NF4

Table 1: Comparison of experimental settings between our reproduction and the original TRAYER paper.

conception statements are treated as candidate extractions for further review.

Misconception-conditioned re-tutoring. We re-run the tutoring dialogues with the mined misconceptions appended to the tutor agent’s system prompt (e.g., “*The student may hold the following misconceptions: ...*”). All other settings—turn limit, student model, decoding parameters—are held fixed. We then recompute $\text{pass}@k$ and $\text{recall}@k$ on the resulting student solutions and compare against the baseline to measure the effect of misconception conditioning.

5 Experiments and Results

5.1 Experimental Setup

We evaluate on 4 repositories from EvoCodeBench spanning 22 tasks total: **EasyVolcap** (12 tasks, volumetric video capture), **searcharray** (6 tasks, information retrieval), **UHGEval/xinhua** (1 task, hallucination evaluation), and **stable-diffusion-webui-forge** (3 tasks, image generation). Each task is evaluated at three student proficiency levels: *low_level* (function signature only), *med_level* (+ dependency paths), and *high_level* (+ surrounding code context).

For each level, we generate posttest prompts from each of the 8 dialogue rounds and produce $N=10$ code completions per prompt. $\text{Pass}@k$ is computed using the unbiased estimator from Chen et al. (2021).

5.2 Baseline Results

Table 2 shows the best-round $\text{Pass}@1$ and $\text{Pass}@10$ for each project and level, compared to the paper’s reported values on the same repositories.

Simple tasks. On sd-forge and UHGEval, our system dramatically exceeds the paper (86–100% vs. 20–83% $\text{Pass}@1$). These tasks involve short,

Project	Level	Paper		Ours	
		P@1	P@10	P@1	P@10
sd-forge	low	43.3	100	86.7	100
	med	83.3	100	90.0	100
	high	40.0	100	86.7	100
UHGEval	low	20.0	100	100	100
	med	40.0	100	90.0	100
	high	30.0	100	90.0	100
searcharray	low	3.3	16.7	0.0	0.0
	med	5.0	16.7	0.0	0.0
	high	0.0	0.0	15.0	25.0
EasyVolcap	low	23.3	33.3	23.0	33.3
	med	14.2	25.0	16.0	30.0
	high	23.3	33.3	15.8	33.3

Table 2: Best-round Pass@ k (%) per project and level. Bold indicates the higher value. Our results use the best-performing round for each project-level combination.

well-defined functions (model setup, JSON aggregation) where even minimal tutoring context is sufficient. All dialogues terminated early via the verifier’s early-stop mechanism, typically after 1–2 rounds.

Complex tasks. On searcharray, we observe 0% Pass@1 at low and med levels across all 8 rounds, consistent with the paper. However, at high level, we observe a notable improvement: Pass@1 rises from 0% at R1–R3 to 15% at R8, exceeding the paper’s 0%. This suggests that additional tutoring rounds provide genuine value for complex tasks with sufficient context.

EasyVolcap. Results closely match the paper. At low level, Pass@1 improves from 12.5% (R1) to 23.0% (R7), nearly matching the paper’s 23.3%. At med level, we exceed the paper (16.0% vs. 14.2%). High level trails slightly (15.8% vs. 23.3%) but matches Pass@10.

5.3 Aggregate Comparison

Table 3 shows the task-weighted average Pass@ k across all 22 tasks, using each project-level’s best round.

Overall, our baseline outperforms the paper by **+6.8 percentage points on Pass@1** (27.5% vs. 20.8%) while maintaining comparable Pass@10 (38.0% vs. 37.9%). The improvement is largest at high level (+9.1pp), suggesting our configuration benefits most when the student has access to richer code context.

Level	Pass@1		Pass@10	
	Paper	Ours	Paper	Ours
low	20.5	28.9	40.9	36.4
med	22.3	25.1	36.4	34.5
high	19.5	28.6	36.4	43.2
Avg	20.8	27.5	37.9	38.0

Table 3: Task-weighted average Pass@ k (%) across all 22 tasks. Our system outperforms the paper by +6.8 percentage points on Pass@1.

Round	Low	Med	High
R1	12.5	3.3	15.8
R2	6.0	8.0	15.0
R3	4.0	10.0	12.0
R4	0.0	16.0	9.0
R5	15.0	8.0	8.0
R6	16.0	8.0	5.0
R7	23.0	10.0	6.0
R8	21.0	10.0	6.0

Table 4: EasyVolcap Pass@1 (%) by round and level. Low-level students improve with more tutoring (R1: 12.5% → R7: 23.0%), while high-level students degrade (R1: 15.8% → R8: 6.0%).

5.4 Round-by-Round Analysis

Table 4 illustrates how Pass@1 evolves across tutoring rounds for EasyVolcap, our largest project (12 tasks).

Two opposing trends emerge:

- **Tutoring helps low-level students.** Pass@1 nearly doubles from R1 to R7 (12.5% → 23.0%), as the tutor gradually fills knowledge gaps.
- **Over-tutoring hurts high-level students.** Pass@1 degrades from R1 to R8 (15.8% → 6.0%), suggesting that additional rounds introduce noise or conflicting guidance for students who already have sufficient context.

This pattern motivates our misconception-conditioned approach: rather than providing more tutoring rounds indiscriminately, the tutor should be informed about *what* the student is getting wrong and address it directly.

5.5 McMiner Results

McMiner analysis on TRAVER outputs. We ran McMiner on TRAVER’s generated student code and accompanying round/problem descriptions across all tutoring rounds and difficulty levels

TRAVER source	Files	Rounds	Code samples	Share
codeformer_model	3	3	30	0.8%
easyvolcap	24	246	2,460	67.0%
gfpgan_model	5	8	80	2.2%
searcharray	24	106	1,060	28.9%
xinhua	4	4	40	1.1%
Total	60	367	3,670	100.0%

Table 5: TRAVER outputs used as input to McMiner.

Run	Unit	Inputs	Parsed	Rate
McMiner-S	Code sample	3,670	3,665	99.86%
McMiner-M (initial)	Group	220	80	36.36%
McMiner-M (fixed)	Group	220	220	100.00%
McMiner-M (S-filtered)	Selected group	20	20	100.00%

Table 6: McMiner parsing results using local Mistral-7B-Instruct-v0.2.

to track how candidate misconceptions evolved as solutions changed over time. Table 5 summarizes the TRAVER outputs used as input to McMiner. Because a Gemini run failed near completion and commercial API models were costly, we used the local Mistral-7B-Instruct-v0.2 model as our baseline mining model.

As shown in Table 6, McMiner-S successfully parsed 3,665 out of 3,670 code-sample outputs, achieving a 99.86% parse success rate. McMiner-M initially parsed only 80 out of 220 misconception groups because many grouped prompts exceeded the local model’s context window. After limiting each McMiner-M prompt to exactly two code examples per group, the corrected run successfully parsed all 220 groups.

These runs produced a large set of candidate misconceptions, motivating us to explore how misconception-aware signals could be incorporated directly into the tutoring pipeline to improve dialogue guidance from the beginning of the interaction.

5.6 Misconception-Conditioned Re-tutoring

We evaluate the effect of conditioning the tutor on McMiner-detected misconceptions. We compare three conditions on EasyVolcap (12 tasks), our largest and most diagnostically rich project:

- **Baseline:** Standard TRAVER tutoring (no misconception information).
- **McMiner:** McMiner-in-the-loop tutoring, where detected misconceptions are injected into the tutor’s system prompt each round.
- **Clean:** Same as McMiner, but misconception-related jargon is stripped from the posttest

Level	Baseline	McMiner	Clean
low	23.0 (R7)	11.0 (R6)	10.8 (R1)
med	16.0 (R4)	16.7 (R8)	17.5 (R8)
high	15.8 (R1)	24.2 (R7)	31.7 (R7)

Table 7: EasyVolcap best-round Pass@1 (%) across conditions. McMiner-Clean achieves +15.9pp over the baseline’s best round at high level, but degrades low-level performance by 12pp.

Metric	Baseline	McMiner	Clean	$\Delta_{BL \rightarrow CL}$
Pass@1	6.0	24.2	31.7	+25.7
Pass@3	12.2	35.7	40.8	+28.6
Pass@5	15.0	39.6	41.6	+26.6
Pass@10	20.0	41.7	41.7	+21.7
Best-round baseline: 15.8% P@1 (R1) $\rightarrow \Delta = +15.9$				

Table 8: EasyVolcap high_level Round 7: Pass@k (%) comparison. All values are from the same round (R7). The Δ column shows the same-round gap; comparing against the baseline’s best round (15.8% at R1) yields +15.9pp.

prompts before code generation, retaining only code-relevant guidance.

EasyVolcap results. Table 7 shows the best-round Pass@1 for each condition and level.

The effect of misconception conditioning is **strongly level-dependent**. At high level, McMiner-Clean achieves 31.7% Pass@1 at R7—a **+15.9 percentage point improvement** over the baseline’s best round of 15.8% (R1). Comparing at the same round (R7), the gap is even larger: the baseline drops to 6.0% while McMiner-Clean reaches 31.7% (+25.7pp), as shown in Table 8.

However, at low level, both McMiner and Clean **degrade performance** by approximately 12 percentage points relative to the baseline. Both conditions degrade identically, confirming that the damage originates from the tutoring dialogue content itself—not from misconception jargon in the posttest prompts. Low-level students produce code with fundamental errors (e.g., missing imports, wrong function signatures), which McMiner misinterprets as conceptual misconceptions, causing the tutor to focus on the wrong issues.

At med level, the three conditions perform comparably, with Clean showing a marginal edge (+1.5pp over baseline at R8). The clean-prompt ablation provides a small but consistent benefit in early rounds (R1: 7.5% vs. baseline 3.3%) and late rounds (R8: 17.5% vs. baseline 10.0%).

Project	Level	BL	MC	Clean
sd-forge	low	86.7	100	—
	med	90.0	100	—
	high	86.7	100	—
UHGEval	low	100	100	—
	med	90.0	100	—
	high	90.0	100	—
searcharray	low	0.0	0.0	0.0
	med	0.0	0.0	0.0
	high	15.0	11.7*	0.0
EasyVolcap	low	23.0	11.0	10.8
	med	16.0	16.7	17.5
	high	15.8	24.2	31.7

*McMiner-Thorough variant (12 rounds + dedup).

Table 9: Best-round Pass@1 (%) across all projects and conditions. McMiner maintains or improves performance on easy tasks (sd-forge, UHGEval) and improves EasyVolcap at high level, but degrades searcharray and EasyVolcap low level. “—” indicates the Clean ablation was not run.

Cross-project comparison. Table 9 shows best-round Pass@1 for all projects under the three conditions.

On sd-forge and UHGEval, McMiner achieves 100% Pass@1 across all levels—matching or exceeding the baseline. These tasks involve short, well-defined functions where misconception-aware tutoring does not interfere with code quality.

On searcharray (6 tasks), all McMiner conditions produce 0% Pass@1 across all levels and rounds, compared to the baseline’s 15.0% at high level R8. We also evaluated an extended “Thorough” configuration with semantic deduplication and 12 rounds; McMiner-Thorough achieves 11.7% Pass@1 at high level R1 (1/6 tasks), but this remains below the baseline. searcharray tasks require precise knowledge of an unfamiliar API (roaring bitmaps, custom postings lists), where misconception detection provides limited value.

Clean-prompt ablation. The consistent advantage of Clean over raw McMiner on EasyVolcap reveals an important finding about prompt design: misconception-related pedagogical language (e.g., “I notice your code has a fundamental error in how you handle...”) consumes context budget that would otherwise contain actionable code hints. For high-level students who already understand the API surface, removing this meta-discussion and retaining only code-focused guidance yields a +7.5pp improvement over raw McMiner at R7 (31.7% vs. 24.2%).

6 Conclusion

We presented an approach that combines misconception detection with dialogue-based tutoring for programming education. Our baseline reproduction of TRAVER on 4 repositories (22 tasks) demonstrates that the framework produces meaningful tutoring outcomes, with an overall Pass@1 of 27.5% compared to the paper’s average of 20.8% (+6.8pp).

Our round-by-round analysis reveals an important asymmetry: low-level students benefit from extended tutoring (Pass@1 nearly doubles over 7 rounds), while high-level students show diminishing returns or degradation. This finding motivates our core contribution—integrating McMiner’s misconception detection into the tutoring loop to provide targeted, concept-focused guidance rather than indiscriminate additional rounds.

Limitations

Our evaluation is limited to 4 of the 25 EvoCodeBench repositories (22 out of 100 tasks) due to storage and environment constraints. Additionally, we substitute Mistral-7B for the original Mixtral-8x7B student model and use 4-bit quantization for local models, which may affect generation quality. The McMiner-conditioned re-tutoring experiments are still in progress and will be included in the final version.

Integrating McMiner’s automated misconception discovery with TRAVER’s tutoring agent also incurs significant computational overhead. Running multiple reasoning-oriented large language models requires substantial financial and hardware resources, which may limit large-scale deployment in resource-constrained educational environments. Local deployment is further constrained by older GPU architectures, such as V100 GPUs, which lack support for newer quantized kernels required by many efficient open-weight models.

From a pedagogical perspective, automated misconception detection introduces the risk of premature intervention. If the tutor immediately corrects a student’s misconception, it may reduce opportunities for productive struggle, self-correction, and independent debugging, which are important aspects of programming education. Over-reliance on automated tutoring systems may also reduce instructors’ own diagnostic engagement with students.

Methodologically, our pipeline relies on LLM-simulated students through the DICT evaluation

protocol. While scalable for benchmarking, simulated learners may not accurately reflect the complex reasoning processes and behaviors of real students in authentic educational settings. Furthermore, our implementation assumes that a given collection of student code reflects a single dominant misconception, whereas real students often exhibit multiple interacting misconceptions simultaneously.

Finally, both EvoCodeBench and McMiner are limited to Python-based tasks. As a result, the ability of our combined tutoring pipeline to generalize to other programming languages remains unexplored.

Acknowledgments

Claude was used to refine the language of the paper and in generation of some scripts.

References

- Erfan Al-Hossami and Razvan Bunescu. 2026. Mcmining: Automated discovery of misconceptions in student code. *arXiv preprint*.
- Neil C. C. Brown and Amjad Altadmri. 2017. [Novice java programming mistakes: Large-scale data vs. educator beliefs](#). *ACM Trans. Comput. Educ.*, 17(2).
- Mark Chen and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Yulin Chen, Ning Ding, Hai-Tao Zheng, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2024. [Empowering private tutoring by chaining large language models](#). In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM '24*, page 354–364, New York, NY, USA. Association for Computing Machinery.
- Nigel J. Francis, Sue Jones, and David P. Smith. 2025. [Generative ai in higher education: Balancing innovation and integrity](#). *British Journal of Biomedical Science*, Volume 81 - 2024.
- W. Lewis Johnson and Elliot Soloway. 1984. Proust: Knowledge-based program understanding. In *Proceedings of the 7th International Conference on Software Engineering, ICSE '84*, page 369–380. IEEE Press.
- Deuksin Kwon, Emily Weiss, Tara Kulshrestha, Kushal Chawla, Gale Lucas, and Jonathan Gratch. 2024. [Are LLMs effective negotiators? systematic evaluation of the multifaceted capabilities of LLMs in negotiation dialogues](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 5391–5413, Miami, Florida, USA. Association for Computational Linguistics.

et al. Li. 2024. Evocodebench: Benchmarking repository-level code generation. *arXiv preprint*.

Zewei Tian, Alex Liu, Lief Esbenshade, Shawon Sarkar, Zachary Zhang, Kevin He, and Min Sun. 2025. [Implementation considerations for automated AI grading of student work](#). In *Proceedings of the Artificial Intelligence in Measurement and Education Conference (AIME-Con): Full Papers*, pages 9–20, Wyndham Grand Pittsburgh, Downtown, Pittsburgh, Pennsylvania, United States. National Council on Measurement in Education (NCME).

Jian Wang and 1 others. 2025. Training turn-by-turn verifiers for dialogue tutoring agents. In *ACL*.

Rose Wang, Qingyang Zhang, Carly Robinson, Susanna Loeb, and Dorottya Demszky. 2024. [Bridging the novice-expert gap via models of decision-making: A case study on remediating math mistakes](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2174–2199, Mexico City, Mexico. Association for Computational Linguistics.

Yu Zhang, Xiusi Chen, Bowen Jin, Sheng Wang, Shuiwang Ji, Wei Wang, and Jiawei Han. 2024. [A comprehensive survey of scientific large language models and their applications in scientific discovery](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8783–8817, Miami, Florida, USA. Association for Computational Linguistics.

Yingxiu Zhao, Bowen Yu, Binyuan Hui, Haiyang Yu, Minghao Li, Fei Huang, Nevin L. Zhang, and Yongbin Li. 2024. [Tree-instruct: A preliminary study of the intrinsic relationship between complexity and alignment](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 16776–16789, Torino, Italia. ELRA and ICCL.

A Per-Project Round-by-Round Results

This appendix provides complete Pass@ k ($k \in \{1, 3, 5, 10\}$) results for all projects, levels, and tutoring rounds. BL = Baseline, MC = McMiner-Loop, CL = McMiner-Clean.

A.1 sd-forge (3 tasks) and UHGEval (1 task)

These projects are essentially solved: all conditions achieve near-perfect scores. Table 10 reports the baseline alongside McMiner-Loop results from the corrected Colab evaluation.

Analysis. These four tasks—codeformer_model.setup_model (model initialization), gfpagan_model.gfpagan_fix_faces and gfpagan_model.setup_model

Project	Level	Cond.	P@1	P@3	P@5	P@10	Round	P@1	P@3	P@5	P@10
codeformer	low	BL R1	90.0	100	100	100	R1	0.0	0.0	0.0	0.0
		MC R1–3	100	100	100	100	R2	0.0	0.0	0.0	0.0
	med	BL R1	100	100	100	100	R3	0.0	0.0	0.0	0.0
		MC R1–3	100	100	100	100	R4	7.5	17.7	22.9	25.0
	high	BL R1	90.0	100	100	100	R5	0.0	0.0	0.0	0.0
		MC R1–3	100	100	100	100	R6	12.5	22.9	24.9	25.0
gfgan	low	BL R1	85.0	100	100	100	R7	7.5	17.7	22.9	25.0
		MC R1–3	95–100	100	100	100	R8	15.0	24.2	25.0	25.0
	med	BL R1	85.0	100	100	100					
		MC R1–3	100	100	100	100					
	high	BL R1	85.0	100	100	100					
		MC R1–8	100	100	100	100					
xinhua	low	BL R1	100	100	100	100					
		MC R1–3	100	100	100	100					
	med	BL R1	90.0	100	100	100					
		MC R1–3	100	100	100	100					
	high	BL R1	90.0	100	100	100					
		MC R1–3	100	100	100	100					

Table 10: sd-forge and UHGEval Pass@ k (%). McMiner matches or exceeds baseline across all rounds (all rounds identical; range shown). BL early-stopped at R1–R3; MC continued for 3–8 rounds.

(face restoration setup), and xinhua.XinhuaHallucinations.statistics (JSON aggregation)—involve short, well-defined functions (1–10 lines) with clear signatures and minimal API surface. The baseline already achieves 85–100% P@1 at R1, indicating that even a single tutoring round provides sufficient context for the student model to generate correct completions. McMiner maintains or slightly improves these scores: misconception-aware tutoring neither helps nor harms when the underlying task is simple enough that the code generation model can solve it from the function signature alone. The baseline conversations early-stop after 1–3 rounds because the tutor recognizes the student has converged, while McMiner conversations extend to 3–8 rounds due to the misconception injection prompting additional dialogue turns. Despite these extra rounds, performance remains saturated at 100%, confirming that McMiner does not introduce harmful noise for easy tasks.

A.2 searcharray (6 tasks)

A.2.1 Baseline (R1–R8)

Low and med levels produce 0% on all metrics across all 8 rounds. Table 11 shows the high_level detail.

Baseline analysis. searcharray is the hardest project in our benchmark. Its 6 tasks require knowledge of roaring bitmaps, custom postings

Table 11: searcharray baseline high_level Pass@ k (%). The only task that passes is searcharray.solr.edismax. Low and med = 0% for all rounds.

lists, and the searcharray library’s internal API—none of which appear in standard training data.

At low and med levels, no task passes across any round: the simulated students produce code with wrong attribute names (e.g., positions_dict instead of the correct accessor), import nonexistent libraries (anserini), or trigger infinite loops in while-based bit counting. At high level, a single task (searcharray.solr.edismax) begins passing at R4, reaching 15% P@1 by R8. This task involves query parsing logic that is more amenable to iterative refinement through tutoring. The intermittent zero at R5 (between non-zero R4 and R6) reflects the stochastic nature of code generation: the model occasionally regresses before recovering in subsequent rounds. The gap between P@1 (15%) and P@10 (25%) at R8 indicates that correct solutions exist in the completion distribution but are not consistently the top-ranked output.

A.2.2 McMiner-Thorough (R1–R12)

Low and med levels remain 0% for all 12 rounds. Table 12 shows the high_level detail.

Round	P@1	P@3	P@5	P@10
R1	11.7	16.5	16.7	16.7
R2	5.0	11.8	15.3	16.7
R3–R12	0.0	0.0	0.0	0.0

Table 12: searcharray McMiner-Thorough high_level Pass@ k (%). Best result is 11.7% P@1 at R1 (1/6 tasks), below baseline’s 15.0% at R8.

McMiner-Thorough analysis. The Thorough configuration applies semantic deduplication to misconception prompts and extends tutoring to 12 rounds with accumulated context. Despite these enhancements, searcharray remains largely unsolved. The best result—11.7% P@1 at high_level R1—

solves the same single task (edismax) as the baseline, but with lower probability. Critically, performance degrades to 0% by R3 and never recovers through R12, whereas the baseline improves from R4 to R8. This suggests that McMiner’s misconception injection actively interferes with the tutoring trajectory for API-heavy tasks: the tutor spends dialogue turns addressing perceived conceptual misunderstandings when the real bottleneck is the student model’s lack of domain-specific API knowledge. Extended rounds (R9–R12) provide no recovery, confirming that more tutoring cannot compensate for fundamentally misaligned misconception feedback.

A.3 EasyVolcap (12 tasks) — Full 3-Way Comparison

A.3.1 low_level

Low-level analysis. McMiner consistently degrades low_level performance, with the gap widening as rounds increase. At R1, the three conditions are nearly identical (12.5/10.8/10.8% P@1), but by R7 the baseline reaches 23.0% while both McMiner (10.0%) and Clean (8.0%) plateau around 10%. This divergence pattern is consistent across all Pass@k values: at R7, the baseline achieves 27.1/29.2/30.0% (P@3/5/10) while McMiner and Clean are stuck at 10%. The root cause is that low-level students produce syntactically broken diagnostic code that McMiner misinterprets as conceptual misconceptions. For example, a student who writes `self.data = None` (a placeholder) triggers a misconception about “fundamental misunderstanding of data initialization,” causing the tutor to spend subsequent rounds correcting a non-existent conceptual error rather than guiding the student toward the correct API usage. Crucially, the Clean ablation provides no benefit over raw McMiner at this level (~1pp difference), confirming that the damage originates in the *tutoring dialogue content itself*, not in residual misconception jargon in the posttest prompt. The baseline’s R7 peak (23.0%) followed by slight decline at R8 (21.0%) suggests an optimal tutoring window of 6–7 rounds for low-level students.

A.3.2 med_level

Med-level analysis. The med_level results exhibit a distinctive two-phase pattern. In *Phase 1* (R1–R5), the baseline generally outperforms McMiner conditions, with the baseline peaking at 16.0% P@1 at R4 while McMiner collapses to

0.0% and Clean to 1.7% at the same round. The R4 collapse is particularly striking: it coincides with the point where McMiner misconception feedback accumulates enough to derail the tutoring trajectory, while the baseline tutor continues productive instruction. In *Phase 2* (R6–R8), the relationship inverts: McMiner conditions recover and surpass the baseline, with Clean reaching 17.5% P@1 at R8 versus the baseline’s 10.0%. This recovery suggests that after enough rounds, the accumulated tutoring context overcomes the initial misconception-induced noise. Clean consistently outperforms raw McMiner by 0.8–4.2pp at P@1 across most rounds, with the advantage most pronounced at R1 (7.5% vs. 2.5%). This pattern holds across all Pass@k: at R1, Clean achieves 17.3/22.3/25.0% (P@3/5/10) versus McMiner’s 6.9/10.6/16.7%, indicating that removing misconception jargon from the posttest prompt frees context budget for code-relevant hints. The net effect is roughly neutral: baseline best (16.0% at R4) versus Clean best (17.5% at R8) yields a modest +1.5pp gain.

A.3.3 high_level

High-level analysis. This is our headline finding. The high_level results show a dramatic and consistent advantage for McMiner conditions, with the effect amplifying through later rounds. The baseline peaks at R1 (15.8% P@1) and degrades steadily to 6.0% by R7—a pattern we attribute to “over-tutoring,” where additional dialogue rounds introduce noise and conflicting guidance for students who already understand the API surface. McMiner reverses this trajectory: performance *improves* from 15.8% (R1) to 24.2% (R7), a +8.4pp gain over the same-round baseline. Clean amplifies this further to 31.7% at R7, yielding a +25.7pp same-round delta.

The Clean advantage over raw McMiner (31.7% vs. 24.2% at R7, +7.5pp) reveals an important prompt engineering insight: misconception-related pedagogical language (e.g., “I notice your code has a fundamental error in how you handle...”) consumes the 60-word posttest prompt budget. For high-level students who already understand the concepts, this meta-discussion displaces actionable code hints. Stripping it leaves more room for concrete API guidance that the code generation model can directly leverage.

The improvement is consistent across all Pass@k. At R7, Clean achieves 40.8% P@3, 41.6% P@5, and 41.7% P@10, versus baseline’s

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	12.5	10.8	10.8	22.2	15.3	17.8	27.1	19.0	23.1	33.3	25.0	33.3
R2	6.0	4.0	5.0	15.4	8.3	11.3	21.9	9.8	14.8	30.0	10.0	20.0
R3	4.0	7.0	6.0	10.1	17.2	15.4	14.2	23.3	21.9	20.0	30.0	30.0
R4	0.0	2.0	2.0	0.0	5.3	5.3	0.0	7.8	7.8	0.0	10.0	10.0
R5	15.0	7.0	8.0	19.7	9.9	10.0	20.0	10.0	10.0	20.0	10.0	10.0
R6	16.0	11.0	10.0	20.0	13.0	10.0	20.0	15.0	10.0	20.0	20.0	10.0
R7	23.0	10.0	8.0	27.1	10.0	10.0	29.2	10.0	10.0	30.0	10.0	10.0
R8	21.0	5.0	4.0	23.0	9.2	8.3	25.0	10.0	9.8	30.0	10.0	10.0

Table 13: EasyVolcap low_level: full Pass@ k (%) for all conditions. Baseline dominates consistently from R5 onward across all metrics.

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	3.3	2.5	7.5	8.9	6.9	17.3	13.0	10.6	22.3	16.7	16.7	25.0
R2	8.0	5.0	8.3	16.2	11.4	15.0	19.1	14.6	16.5	20.0	16.7	16.7
R3	10.0	5.0	3.3	15.3	8.1	6.9	17.8	8.3	8.1	20.0	8.3	8.3
R4	16.0	0.0	1.7	23.7	0.0	4.4	27.5	0.0	6.5	30.0	0.0	8.3
R5	8.0	2.5	4.2	15.7	5.9	7.6	20.0	7.6	8.3	30.0	8.3	8.3
R6	8.0	7.5	7.5	10.0	8.3	8.3	10.0	8.3	8.3	10.0	8.3	8.3
R7	10.0	15.0	16.7	10.0	16.7	16.7	10.0	16.7	16.7	10.0	16.7	16.7
R8	10.0	16.7	17.5	13.0	16.7	19.2	15.0	16.7	20.8	20.0	16.7	25.0

Table 14: EasyVolcap med_level: full Pass@ k (%) for all conditions. Roughly neutral; Clean has an edge at R1 and R7–R8.

12.2/15.0/20.0%. The narrowing gap between P@1 and P@10 for Clean (31.7% vs. 41.7%) compared to baseline (6.0% vs. 20.0%) indicates that McMiner-Clean not only increases the probability of correct solutions but also concentrates them toward the top of the generation distribution.

The R8 dip in Clean (15.0% vs. McMiner’s 21.7%) is likely attributable to variance at the tutoring ceiling: by R8, most improvable tasks have already been solved, and the remaining unsolved tasks are resistant to further tutoring.

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	15.8	15.8	19.2	25.0	28.0	28.8	28.5	31.4	31.4	33.3	33.3	33.3
R2	15.0	13.3	16.7	25.4	26.0	26.0	28.9	30.8	29.0	30.0	33.3	33.3
R3	12.0	15.0	10.0	20.1	24.9	17.2	24.2	29.6	21.3	30.0	33.3	25.0
R4	9.0	15.0	16.7	16.8	21.1	23.5	19.2	23.1	24.8	20.0	25.0	25.0
R5	8.0	23.3	22.5	12.9	27.4	31.2	15.0	29.2	33.1	20.0	33.3	33.3
R6	5.0	21.7	22.5	11.3	29.2	28.7	14.8	31.5	31.4	20.0	33.3	33.3
R7	6.0	24.2	31.7	12.2	35.7	40.8	15.0	39.6	41.6	20.0	41.7	41.7
R8	6.0	21.7	15.0	9.7	24.7	19.1	10.0	25.0	20.8	10.0	25.0	25.0

Table 15: EasyVolcap high_level: full Pass@ k (%) for all conditions. McMiner-Clean dominates from R4 onward across all metrics. At R7, Clean achieves 31.7%/40.8%/41.6%/41.7% (P@1/3/5/10) vs. baseline’s 6.0%/12.2%/15.0%/20.0%.